Junior AI Developer Assessment: ResolveBot

A customer support chat assistant for an online store

1. Overview

You will build a chat assistant for **ShopSphere**, a made-up online electronics store. A customer who is already logged in sends it messages about their orders. The assistant can:

- look up the customer's orders,
- answer questions from the store's policy documents,
- work out if an item can be returned,
- open a return or a support ticket, **but only after the customer says yes**.

Item	Detail
Time limit	**7 calendar days** from the time you receive this assignment
Language	Python 3.10 or later
Frameworks and libraries	Use **any Python-based** framework or library (for example FastAPI, Flask, LangChain, LlamaIndex), or none
LLM	Any provider: OpenAI, Anthropic, Google Gemini or a local model
Interface	**No user interface is needed. The API endpoint alone is enough.**
Tests	**Automated tests are not required.**

What we value. Working code is more important than many features. A small assistant that runs and follows the rules scores higher than a big one that half works. Tell us honestly what does not work. You may use AI coding assistants; say where in your README. You will explain your code to us later, so make sure you understand it. If something is unclear, make a sensible choice and write it under "Assumptions" in your README.

Words used in this document

LLM	The AI model you call (for example GPT, Claude or Gemini).
Agent	The LLM, when it decides by itself which tool to use next. Here there is **one** agent: the support assistant.
Tool	A Python function that the agent can ask to run, such as "look up this order". Your code runs it and gives the result back to the agent. There are **five** tools.
RAG	"Retrieval": search the policy file for the few paragraphs that answer the question, and give only those to the LLM.
Route	A label on every reply that says what kind of reply it is: `answer`, `clarify`, `no_answer` or `out_of_scope`.
Proposal	The assistant says "I can do X for you. Shall I go ahead?" and waits. Nothing is saved until the customer says yes.
`APP_TODAY`	The date your code must treat as "today" (default 1 July 2026). Do not use the computer's clock.

What you are given

A `data/` folder. When we test, we swap it for our own copy with the same file names but different values.

- `policies.md`: all 10 policy documents in **one Markdown file**. Each document starts with a `#` title line, then a `Document ID` line. Sections have numbers like `§2.1`, which you will quote.

- `manifest.json`: one entry per document: `doc_id`, `title`, `doc_type` (policy or faq), `effective_date` (the first day it applies) and `status` (current or superseded, which means replaced by a newer version).
- `shopsphere.db`: a SQLite database with these tables:

Table	Main columns
customers	customer_id, full_name, email, city
products	product_id, name, category, price, returnable (yes / unopened_only / no), warranty_class (standard / extended / none)
orders	order_id (like SS-10458), customer_id, order_date, status (processing / shipped / delivered / cancelled), total
order_items	item_id (like SS-10458-1), order_id, product_id, qty, unit_price, opened (1 = opened)
shipments	order_id, carrier, status, delivered_at, eta
policy_parameters	key, value. All the business numbers, for example `return_window_days` = 30.
return_requests	Empty. Your code adds rows: request_id, order_id, customer_id, item_ids, reason, status (requested), created_at
support_tickets	Empty. Your code adds rows: ticket_id, order_id, customer_id, ticket_type, summary, status (open), created_at

Important: read numbers such as "30 days" or "12 months" from `policy_parameters`. **Never type them into your code**, because our test copy uses different numbers. Do not change or delete the existing tables or columns (you may add new ones).

2. The Task

Build a small web service on port 8000 with one endpoint, `POST /chat`. Each request carries the customer's ID in the header `X-Customer-Id`.

2.1 What the assistant must handle

Customer wants	Example message	What the assistant does
Track an order	"Where is order SS-10512?"	Looks up the order and gives its status and expected delivery date.
Return an item	"I want to return the speaker from SS-10458."	Checks the return rules, proposes a return, waits for "yes".
Warranty help	"My phone stopped charging. Is it covered?"	Checks the warranty. If the customer asks, proposes a warranty ticket and waits for "yes".
Cancel an order	"Please cancel order SS-10530."	Checks the order has not shipped, proposes a cancellation ticket, waits for "yes".
Ask about a policy	"How many days do I have to return something?"	Searches the policies and answers, quoting the section.

It must also: ask a question back when something is missing (for example "return my charger" when the customer has chargers in three orders), handle an order that does not exist or belongs to someone else, and politely refuse topics that are not about the store. **Not needed:** a user interface, login, real payments, refund amounts, emails, passing the chat to a human, or exchanges.

2.2 The five tools

Use exactly these names, because our tests look for them.

Tool	Inputs	What it does
`get_orders`	order_id (optional), status (optional)	Reads. Returns the customer's orders, their items and delivery details.
`search_policy`	query	Reads. Returns the best matching policy paragraphs, each with its doc_id and section.
`check_return_eligibility`	order_id, item_ids (a list), reason (`changed_mind` or `defective`)	Reads. Checks if the items can be returned (2.3). Returns "eligible" or an error code.
`create_return_request`	order_id, item_ids, reason	Writes a row to return_requests.
`create_support_ticket`	order_id, ticket_type (`warranty` or `cancellation`), summary	Writes a row to support_tickets.

Rules for all tools

- Describe each tool and its inputs clearly so the LLM knows when to use it. If an input is wrong, return an error such as `{"error_code": "INVALID_ITEM", "message": "Item SS-10458-9 is not in this order"}`. Do not crash.
- **Safety:** always take the customer ID from the `X-Customer-Id` header, **never from the LLM**. Every tool that gets an order_id must check in Python that the order belongs to this customer.
- If the order does not exist, **or** belongs to another customer, give the same error, `ORDER_NOT_FOUND`, and reply exactly: "I couldn't find that order on your account." (route `no_answer`). This way nobody can find out that someone else's order exists.
- `get_orders` should also return values your code works out: the delivery date, the number of days since delivery, and for each item the warranty end date and whether the warranty is still active.

Error codes: `ORDER_NOT_FOUND`, `INVALID_ITEM`, `INVALID_REASON`, `INVALID_TICKET_TYPE`, `NOT_DELIVERED`, `NON_RETURNABLE`, `OPENED_NOT_RETURNABLE`, `RETURN_WINDOW_EXPIRED`, `WARRANTY_EXPIRED`, `NO_WARRANTY`, `ORDER_ALREADY_SHIPPED`.

2.3 The business rules (write them in Python)

Your Python code makes every decision. The LLM only explains the result in friendly words. Take all numbers from `policy_parameters`. The examples below use the current values: return window 30 days, warranty 12 or 24 months.

Counting days. Days since delivery = `APP_TODAY` minus the delivery date. "Within 30 days" means 30 or fewer. Example: delivered 1 June, today 1 July: 30 days, so a return is still allowed. On 2 July it is 31 days: too late.

Can the item be returned? The reason is `changed_mind` (the customer just does not want it) or `defective` (it is broken, damaged or the wrong item). Check these in order and stop at the first one that fails:

#	Check	If it fails, return
1	The order exists, belongs to the customer, contains the items and has been delivered	`ORDER_NOT_FOUND`, `INVALID_ITEM` or `NOT_DELIVERED`
2	The product's `returnable` is not `no` (gift cards and software are never returnable)	`NON_RETURNABLE`
3	It is within `return_window_days` of delivery	`RETURN_WINDOW_EXPIRED`
4	**Only for an opened item and `changed_mind`:** the product is not `unopened_only` (for example earbuds)	`OPENED_NOT_RETURNABLE`

If all checks pass, the items are eligible. You do **not** need to work out any refund amount.

Example	Result
Speaker, not opened, changed mind, day 10	Eligible
Speaker, opened, changed mind, day 30	Eligible (day 30 is the last day)
Speaker, not opened, changed mind, day 31	`RETURN_WINDOW_EXPIRED`
Earbuds, opened, changed mind, day 5	`OPENED_NOT_RETURNABLE`
Earbuds, opened, defective, day 5	Eligible
Gift card, not opened, changed mind, day 2	`NON_RETURNABLE`

Warranty. The product's `warranty_class` decides the length: standard = `standard_warranty_months` (12), extended = `extended_warranty_months` (24, used for laptops), none = no warranty. The warranty ends on the same date that many months after delivery, and that last day is still covered. Examples: delivered 15 March 2026 with 12 months: covered until 15 March 2027. Delivered 31 January with 1 month: February has no 31st, so it ends on the last day of February.

> **Tip:** `dateutil.relativedelta(months=n)` from the `python-dateutil` package handles the month arithmetic, including the end-of-month case, for you.

When can the assistant save something? Check again just before saving. If the check fails, do not save; set the tool status to `error` and tell the customer why. If it passes, add one row with a new unique ID (for example `str(uuid.uuid4())`), the current time, and status `requested` (returns) or `open` (tickets).

Action	Allowed only when
Return request	`check_return_eligibility` succeeds for the same order, items and reason.
Warranty ticket	The order is delivered and at least one item is still under warranty.
Cancellation ticket	The order status is `processing`. If it has shipped, return `ORDER_ALREADY_SHIPPED`.

2.4 Searching the policies (the RAG part)

- Your code reads `policies.md` by itself. Split it into the 10 documents at each `#` title, and use `manifest.json` (matched by `doc_id`) to know each document's status and start date.
- Only use a document if its status is `current` **and** its `effective_date` is on or before `APP_TODAY`. **Never use a superseded (old) document.**
- If a policy and the FAQ say different things, follow the policy.
- If a document starts after `APP_TODAY`, it does not apply yet. You may mention it as a future change, with its start date.
- Whenever you use policy text, list it in `citations`, for example `{"doc_id": "returns_policy", "section": "§2.1"}`. List each document you used, and no others.
- If no current document answers the question, use route `no_answer`, say the policies do not cover it, and suggest emailing the support team. Do not guess.
- Send the LLM only the paragraphs it needs, not the whole file. How you split and search is your choice. Do not paste policy facts into your code or prompts.

2.5 How the conversation works

- **Tool loop.** Send the message and tool list to the LLM. If it asks for a tool, run it and send back the result. Repeat until the LLM gives its final reply. Stop after a fixed number of tool calls (for example 6) and reply politely. If one message asks two things, answer both.
- **Ask when unsure.** If the order, item or reason is missing or unclear, reply with route `clarify`. Ask for everything that is missing in one message. If several orders match, list them and ask which one. Never guess.
- **Two steps for any change: propose, then do.**
  1. **Propose** only when the customer asks for the action ("please return it"). If they only ask a question ("can I return it?"), answer and offer to do it. If the rules say no, explain why and propose nothing. If the rules say yes, describe what you

will do (the items and reason, or the ticket type), ask "Shall I go ahead?", set `pending_confirmation` to true, and add the call to `tool_calls` with status `pending_confirmation`. Save nothing yet.
2. **Do it** if the very next message in the same chat is a clear yes ("yes", "confirm", "go ahead", "please do"). Run the tool and report the result.
3. **Anything else:** if the customer changes the details ("just the power bank"), make a new proposal. For any other message, including "no", forget the proposal and reply normally.

Never save anything without a yes, even if the customer says "just do it". Only one proposal can wait at a time.

- **Memory.** Remember each chat's messages and its waiting proposal by `session_id`. A chat belongs to the customer who started it.
- **Dates.** A date without a year means 2026. Write dates clearly, like "3 July 2026".
- **Off-topic** messages (the weather, a poem) get route `out_of_scope` and one line on what you can help with.

2.6 The API

`POST /chat` on port 8000. Headers: `X-Customer-Id` and `Content-Type: application/json`. The caller chooses the `session_id`; a new one starts a new chat. Reply within 30 seconds.

Example: a return in two messages. Customer C1009, same `session_id` both times.

```
Message 1:  {"session_id": "s1", "message": "Return the speaker from SS-10458, I changed my mind."}

Reply 1:
{
  "request_id": "a1b2",
  "answer": "You can return the Nimbra Smart Speaker. It was delivered 10 days ago, which is
             within the 30-day return window. Shall I create the return request?",
  "route": "answer",
  "citations": [{"doc_id": "returns_policy", "section": "§2.1"}],
  "tool_calls": [
    {"name": "get_orders", "args": {"order_id": "SS-10458"}, "status": "success"},
    {"name": "search_policy", "args": {"query": "return window"}, "status": "success"},
    {"name": "check_return_eligibility",
     "args": {"order_id": "SS-10458", "item_ids": ["SS-10458-1"], "reason": "changed_mind"},
     "status": "success"},
    {"name": "create_return_request",
     "args": {"order_id": "SS-10458", "item_ids": ["SS-10458-1"], "reason": "changed_mind"},
     "status": "pending_confirmation"}
  ],
  "pending_confirmation": true
}

Message 2:  {"session_id": "s1", "message": "Yes, go ahead."}

Reply 2:    The return is saved. "tool_calls" shows create_return_request with "status": "success",
            and "pending_confirmation" is false.
```

Return all the fields shown above, with these exact names (you may add more). `tool_calls` lists every tool used for this message, in order. Each reply has one **route**:

Route	When to use it
`answer`	A normal reply: an answer, a proposal, a "no, because...", or the result of a confirmed action.
`clarify`	You need more details from the customer first.
`no_answer`	The order was not found, or the policies do not cover the question. `citations` is empty.
`out_of_scope`	Not about ShopSphere orders or policies. `citations` and `tool_calls` are empty.

HTTP errors (use only these): **400** if the body is not valid JSON or is missing `session_id` or `message`; **401** if `X-Customer-Id` is missing or unknown; **403** if the `session_id` belongs to another customer. Everything else, even "order not found", is a normal 200 reply.

3. What to Submit

3.1 Your repository must contain

Item	What we need
Source code	Easy to read. Keep the tools, the business rules, the policy search and the API in separate files.
`.env.example`	Every setting with a default value pointing at `./data`. No real API keys.
`README.md`	How to install and run; how your solution works (tools, policy search, the yes-step); Assumptions; what does not work yet; a few example chats you tried; where you used AI coding assistants.

3.2 Running it

- One command starts the service on port 8000, for example `uvicorn main:app --port 8000`. Docker is not needed.
- Read these settings from environment variables. We run it with our own API key. Use temperature 0 so answers stay the same each time. Any search method is fine; a local embedding model must run on a normal CPU.

Variable	Default / meaning
`POLICIES_PATH`	`./data/policies.md`
`MANIFEST_PATH`	`./data/manifest.json`
`DB_PATH`	`./data/shopsphere.db`
`APP_TODAY`	`2026-07-01` (the date to treat as today)
`LLM_PROVIDER`	Which LLM provider you use
API key	For example `OPENAI_API_KEY`, `ANTHROPIC_API_KEY` or `GOOGLE_API_KEY`. You may add more settings.

3.3 How to submit

- Share your GitHub repository before the deadline. If you want it private, zip the code and email it to us.
- Do not include the `data/` folder. **Never commit real API keys.**
- We look at the last commit before the deadline, and we may look at your commit history.

3.4 How we check your work

We send test chats to your service using our own data, then read your code and README. **These five checks must pass; if one fails, we stop reviewing.** If you are short on time, make these work first and list what is unfinished in your README.

1. The service starts with the command in your README.
2. Replies have the format shown in 2.6.
3. Nothing is saved to the database without the customer's yes.
4. Return decisions are correct (eligible, or the right error code), even when our data has different numbers.
5. A customer can never see another customer's order.